

# Assignment 7 – Business Logic Layer

---

## Q1. Core Functional Modules of BLL + Interaction with UI

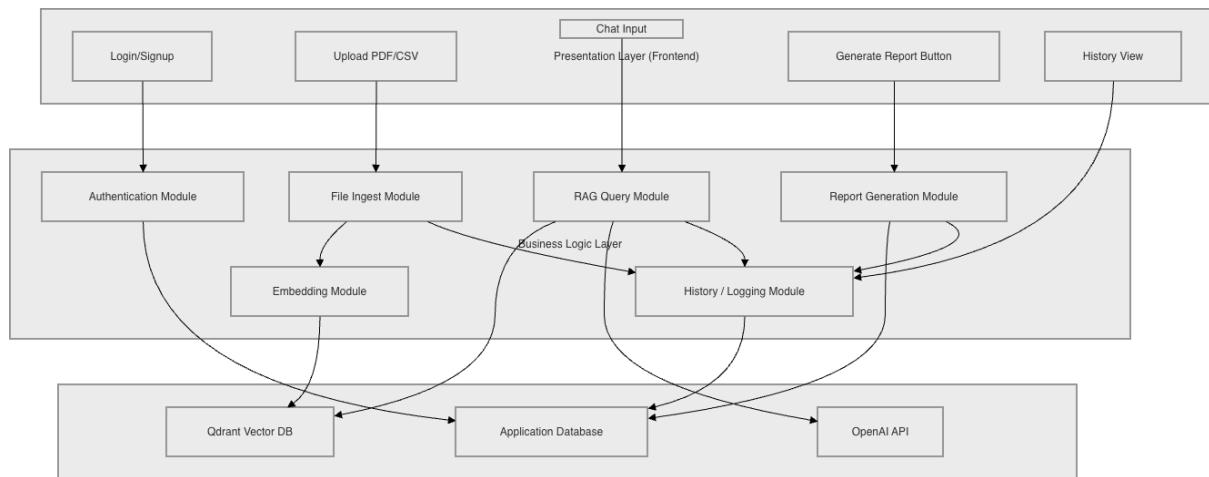
The Business Logic Layer (BLL) sits between the Presentation Layer (UI) and the Data Access Layer (databases). It contains all the core processing logic of InsightFlow.

### Core BLL Modules:

- **Authentication Module** – Handles user signup and login logic
  - **File Ingest Module** – Manages PDF and CSV file processing
  - **Embedding Module** – Converts text chunks into vector embeddings
  - **RAG Query Module** – Processes user queries using retrieval-augmented generation
  - **Report Generation Module** – Processes CSV data and produces structured reports
  - **History / Logging Module** – Records all user actions
- 

### Interaction with UI Components (from Assignment 6)

| UI Component<br>(Presentation Layer) | BLL Module Triggered               | Action                                      |
|--------------------------------------|------------------------------------|---------------------------------------------|
| Signup / Login Form                  | Authentication Module              | Validates credentials, creates session      |
| Upload PDF Button                    | File Ingest Module                 | Extracts text, chunks, sends to Embedding   |
| Upload CSV Button                    | File Ingest Module + Report Module | Reads CSV, cleans data, triggers report     |
| Chat Input Box                       | RAG Query Module                   | Embeds query, retrieves chunks, calls LLM   |
| Generate Report Button               | Report Generation Module           | Processes data, creates downloadable report |
| History View                         | History Module                     | Fetches logged actions from database        |



Caption : *Logical representation of the Business Logic Layer in InsightFlow's microservices architecture, showing how each microservice handles its own business logic between the Presentation Layer (UI) and the Data Access Layer (databases).*

## Q2A. Business Rules Implementation

Business rules are conditions that each BLL module enforces before allowing an operation to proceed.

### Authentication Module

- A user must register with a unique email before logging in
- Password must meet a minimum length requirement
- A user must be authenticated before accessing any feature of the dashboard
- Unauthenticated requests are rejected and redirected to the login page

### File Ingest Module

- Only **.pdf** files are accepted for document upload
- Only **.csv** files are accepted for report generation
- File size must not exceed the defined maximum limit
- Empty or corrupted files are rejected with an error message

### Embedding Module

- Embedding is triggered only after successful text extraction from a valid file
- Empty text chunks are skipped and not sent for embedding
- Each chunk is associated with a user ID and document ID before storage

### **RAG Query Module**

- A PDF document must be uploaded and embedded before a query can be submitted
- User query must not be empty or blank
- Only the authenticated user's documents are searched in the vector database
- If no relevant chunks are found, the system returns a "no context found" response

### **Report Generation Module**

- Report generation proceeds only after a valid CSV file is uploaded
- CSV must contain at least one numeric column for analysis
- Missing or null values in the CSV are handled before processing begins
- Each generated report is linked to the user's session

### **History / Logging Module**

- Every upload, query, and report generation event is automatically logged
- Logs are always associated with the authenticated user

---

## **Q2B. Validation Logic**

Validation ensures all data entering the system is correct and in the proper format before processing.

### **Frontend Validation (Presentation Layer)**

- Email format is validated during signup/login using regex
- Password minimum length is enforced on the signup form
- File type is checked before upload — only **.pdf** or **.csv** is allowed
- Chat input box prevents empty query submission

- CSV file is checked for a `.csv` extension before upload is triggered

### **Backend Validation (BLL Level)**

- File type and size are re-validated at the server (not just the frontend)
- CSV structure is validated — checks that the file has valid headers and at least one data row
- User session token is verified before any BLL module processes a request
- Query text is validated as non-empty before the RAG pipeline begins
- The system confirms a document exists in the vector store before executing a query

### **Example Validation Flow (CSV Upload):**

User selects a file

- Frontend: checks extension is `.csv`
- Frontend: checks file is not empty
- Backend: re-validates file extension
- Backend: reads CSV headers — checks columns exist
- Backend: checks for null/missing values
- If all valid → passes to Report Generation Module
- If invalid → returns error message to UI

### **Example Validation Flow (RAG Query):**

User types a question and clicks Send

- Frontend: checks input is not empty
- Backend: verifies user session is active
- Backend: checks that a document exists for this user
- Backend: validates query is non-empty string
- If valid → passes to RAG Query Module
- If invalid → returns appropriate error to UI

## Q2C. Data Transformation

Data transformation in InsightFlow ensures that data moves between layers in the correct format at each stage of processing.

### Transformation Table:

| Source                          | Transformation Process                | Output                   |
|---------------------------------|---------------------------------------|--------------------------|
| Uploaded PDF                    | Text extraction via PyPDF             | Raw text string          |
| Raw text                        | Chunking (split by size with overlap) | List of text chunks      |
| Text chunks                     | OpenAI Embedding Model                | High-dimensional vectors |
| Vectors                         | Qdrant upsert with metadata           | Stored vector records    |
| User query (text)               | OpenAI Embedding Model                | Query vector             |
| Query vector + retrieved chunks | OpenAI LLM (GPT)                      | Natural language answer  |
| Uploaded CSV                    | Pandas read and clean                 | Structured DataFrame     |
| Cleaned DataFrame               | Statistical summary + OpenAI          | Report content (text)    |
| Report content                  | PDF/DOCX generation library           | Downloadable report file |
| Database records                | JSON serialization                    | API response sent to UI  |

### Explanation of Key Transformations:

- **PDF → Vectors:** When a user uploads a PDF, the File Ingest Module extracts raw text. This text is split into smaller chunks, and each chunk is converted into a numerical vector by the Embedding Module using OpenAI. These vectors represent the semantic meaning of the text and are stored in Qdrant.
  - **Query → Answer:** When a user asks a question, the RAG Module converts the query into a vector, searches Qdrant for the most semantically similar document chunks, and passes those chunks as context to the OpenAI LLM to generate a final answer.
  - **CSV → Report:** When a user uploads a CSV, the Report Generation Module uses Pandas to clean and process the data. Summaries and insights are extracted and passed to OpenAI to generate readable report content, which is then converted into a downloadable PDF or DOCX file.
  - **Database → UI:** All data retrieved from the Application Database (reports, history, user data) is serialized into JSON format before being sent to the frontend for display.
- 

## Conclusion

The Business Logic Layer of InsightFlow consists of six core modules that mediate between the UI and the databases. Each module enforces specific business rules, validates incoming data, and performs the necessary transformations to ensure that the system processes requests correctly and returns reliable outputs to the user.